

Instrukcja wdrożenia CyfroSearch w BitBagSylliusElasticsearchPlugin

Dla kogo: PHP/Symfony developer pracujący z Syllius + BitBag ES Plugin
Cel: Osiągnięcie jakości wyszukiwania z prototypu CyfroSearch (98.9% success rate na top 100 zapytań GA4)
Zakres: Wyszukiwanie produktów + sugestie produktów i kategorii (bez sugestii fraz — w Sylliusie wyświetlamy wyłącznie produkty i kategorie)
Wymagania: Elasticsearch 7.17+, pluginy `analysis-stempel` + `analysis-icu`

Uwaga: Dokument **nie obejmuje** sugestii popularnych fraz wyszukiwania (autosuggest query completions). W Sylliusie okno podpowiedzi prezentuje wyłącznie **sugestie produktów** i **sugestie kategorii (taxonów)** — i dokładnie to jest przedmiotem tego dokumentu.

Spis treści

- 1. Faza 0 — Zmiana .env (0 zmian w kodzie)
- 2. Faza 1 — Pluginy ES + analiza polska
- 3. Faza 2 — Nowe pola w indeksie
- 4. Faza 3 — Intent Detection
- 5. Faza 4 — Nowy ProductsQueryBuilder
- 6. Faza 5 — Nowy FunctionScore
- 7. Faza 6 — Synonimy i analizery
- 8. Faza 7 — Reranking sprzedażowy
- 9. Faza 8 — Testy i walidacja
- 10. Mapa plików — co dodać, co zmienić
- 11. Zapytania testowe
- 12. Docelowa konfiguracja fos_elastica.yaml

Faza 0 — Zmiana .env

Ryzyko: zerowe | Bez zmian w kodzie | Efekt natychmiastowy

Obecne wartości boosterów w `.env` są domyślne (1). Zmiana na wartości przetestowane w prototypie daje natychmiastową poprawę.

Krok 1: Edytuj plik `.env`

```
# --- PRZED (domyślne) ---
ELASTICSEARCH_NAME_BOOSTER=1
ELASTICSEARCH_DESCRIPTION_BOOSTER=1
ELASTICSEARCH_PRODUCER_BOOSTER=1
ELASTICSEARCH_MAIN_CATEGORY_BOOSTER=300
ELASTICSEARCH_SUB_CATEGORY_BOOSTER=200
ELASTICSEARCH_CHILD_CATEGORY_BOOSTER=100
ELASTICSEARCH_POPULARITY_WEIGHT=1
ELASTICSEARCH_NEW_OR_PROMOTION_WEIGHT=1
```

```
ELASTICSEARCH_RATING_WEIGHT=1
ELASTICSEARCH_FEATURED_WEIGHT=100

# --- P0 (zalecane CyfroSearch) ---
ELASTICSEARCH_NAME_BOOSTER=3
ELASTICSEARCH_DESCRIPTION_BOOSTER=1
ELASTICSEARCH_PRODUCER_BOOSTER=3
ELASTICSEARCH_MAIN_CATEGORY_BOOSTER=300
ELASTICSEARCH_SUB_CATEGORY_BOOSTER=200
ELASTICSEARCH_CHILD_CATEGORY_BOOSTER=100
ELASTICSEARCH_POPULARITY_WEIGHT=80
ELASTICSEARCH_NEW_OR_PROMOTION_WEIGHT=20
ELASTICSEARCH_RATING_WEIGHT=10
ELASTICSEARCH_FEATURED_WEIGHT=100
```

Krok 2: Test

Wyszukaj `canon` — aparaty Canon powinny być wyżej niż akcesoria.

Faza 1 — Pluginy ES + analiza polska

Wymaga restartu ES + pełny reindex

Krok 1: Zainstaluj pluginy Elasticsearch

```
# Na serwerze ES:
sudo /usr/share/elasticsearch/bin/elasticsearch-plugin install analysis-stempel
sudo /usr/share/elasticsearch/bin/elasticsearch-plugin install analysis-icu
sudo systemctl restart elasticsearch
```

Uwaga: Jeśli używasz Dockera, dodaj do Dockerfile:

```
FROM docker.elastic.co/elasticsearch/elasticsearch:7.17.5
RUN bin/elasticsearch-plugin install analysis-stempel
RUN bin/elasticsearch-plugin install analysis-icu
```

Krok 2: Zaktualizuj analizery w `fos_elastica.yaml`

W sekcji `settings.analysis` indeksu `bitbag_shop_product` zamień istniejącą konfigurację na:

```
settings:
  analysis:
    char_filter:
      mark_normalizer:
        type: pattern_replace
        pattern: "\\bmark\\s+(i{1,4}v?|iv|v)\\b"
        replacement: "mark$1"
        flags: "CASE_INSENSITIVE"
    filter:
      polish_stem:
        type: stemmer
        language: polish
      icu_folding_filter:
        type: icu_folding
      synonym_graph_filter:
        type: synonym_graph
        synonyms:
          - "fuji, fujifilm, fuij, fuijfilm"
          - "om system, olympus, olimpus"
          - "smallrig, smallring"
          - "canon, kannon, cannon, kanon"
          - "karty pamięci, karta pamięci, karta SD, karty SD"
          - "lustrzanka ⇒ aparat cyfrowy"
          - "leika, lejka, lejca ⇒ leica"
          - "szkło, obiektyw"
          - "statyw, tripod, trójnóg"
          - "lampa, flesz, flash, błysk"
    analyzer:
      polish_full:
        type: custom
        char_filter: [mark_normalizer]
        tokenizer: standard
        filter: [lowercase, polish_stem, icu_folding_filter, synonym_graph_filter]
      polish_prefix:
        type: custom
        tokenizer: standard
        filter: [lowercase, icu_folding_filter, edge_ngram_filter]
      polish_folded:
        type: custom
```

```
char_filter: [mark_normalizer]
tokenizer: standard
filter: [lowercase, icu_folding_filter]
edge_ngram_filter:
  type: edge_ngram
  min_gram: 2
  max_gram: 15
```

Krok 3: Pełny reindex

```
bin/console fos:elastica:populate --env=prod
```

Faza 2 — Nowe pola w indeksie

Wymaga nowych PropertyBuilderów + reindex

Prototyp CyfroSearch używa wielu pól, których brak w obecnym indeksie BitBag. Dodaj je:

Krok 1: Nowe pola w `fos_elastica.yaml` (sekcja properties)

```
properties:
# --- Istniejące pola (zachowaj) ---
enabled: ~
name_pl_pl:
  type: text
  analyzer: polish_full
  fields:
    keyword: { type: keyword }
    sort: { type: icu_collation_keyword, language: pl, country: PL }
    prefix: { type: text, analyzer: polish_prefix }
    folded: { type: text, analyzer: polish_folded }
    morfologik: { type: text, analyzer: polish_full }

# --- Nowe pola do dodania ---
producer:
  type: keyword      # brand name, exact match
ean:
  type: keyword      # EAN barcode
product_code:
  type: keyword      # manufacturer code / SKU
erp_id:
  type: keyword      # Verto ERP ID
description_pl_pl:
  type: text
  analyzer: polish_full
subcategory:
  type: keyword      # nazwa podkategorii (np. "bezlusterkowce")
brand:
  type: keyword      # alias dla producer (do filtrowania)
availability:
  type: keyword      # "in_stock", "na_zamowienie", "out_of_stock"
condition:
  type: keyword      # "new" lub "used"
is_bestseller:
  type: boolean
is_promo:
  type: boolean
is_new:
  type: boolean
has_image:
  type: boolean
price:
  type: float
```

Krok 2: Nowe PropertyBuildery (pliki PHP)

Dla każdego nowego pola utwórz PropertyBuilder w katalogu `PropertyBuilder/`. Wzoruj się na istniejących (np. `ProducerBuilder.php`, `EanBuilder.php`).

Pole ES	Źródło danych	PropertyBuilder
producer	Sylius Product → producer name	Istnieje: <code>ProducerBuilder.php</code>

ean	Sylius Variant → EAN	Istnieje: <code>EanBuilder.php</code>
product_code	Wariant → code / manufacturer_code	Utwórz nowy lub rozszerz istniejący
subcategory	Taxon główny → nazwa podkategorii	Rozszerz <code>ProductTaxonBuilder.php</code>
availability	Stan magazynowy wariantu	Rozszerz <code>InventoryAvailabilityBuilder.php</code>
condition	Atrybut "stan" (nowy/używany)	Rozszerz <code>ProductStateBuilder.php</code>
is_bestseller	Label produktu / atrybut	Utwórz nowy
is_promo	Sylius Promotion	Rozszerz <code>NewPromotionBuilder.php</code>
has_image	Product → hasImages()	Utwórz nowy
price	Sylius ChannelPricing	Utwórz nowy

Krok 3: Zarejestruj w services XML

Dodaj nowe PropertyBuildery do `Resources/services/property_builder.xml`.

Krok 4: Reindex

```
bin/console fos:elastica:populate --env=prod
```

Faza 3 — System detekcji intencji (Intent Detection)

6 nowych plików PHP | Serce logiki wyszukiwania

System Intent Detection analizuje zapytanie użytkownika i wykrywa:

- **Brand intent** — "fuji xt5" → marka Fujifilm, reszta "xt5"
- **Category intent** — "lampa" → 7 podkategorii lamp
- **Condition intent** — "używany canon r5" → filtr na stan "used"
- **Focal-length intent** — "24-70" → filtr na obiektywy
- **Model normalization** — "a7iv" → "a7 iv", "mark III" → "markIII"

Krok 1: Utwórz katalog `IntentDetector/`

Lokalizacja: `src/Component/Elasticsearch/IntentDetector/`

Krok 2: Utwórz 6 plików

Plik 1: `BrandIntent.php` — Value Object

```
<?php
declare(strict_types=1);
namespace App\Component\Elasticsearch\IntentDetector;

final class BrandIntent
{
    public function __construct(
        public readonly string $brand,      // "Fujifilm"
        public readonly string $remainder,  // "xt5"
        public readonly bool $wasAlias,     // true jeśli "fuji" → "Fujifilm"
    ) {}
}
```

Plik 2: `BrandIntentDetector.php`

Wykrywa markę na początku zapytania. Ładuje listę marek z ES przy pierwszym użyciu.

```
<?php
declare(strict_types=1);
namespace App\Component\Elasticsearch\IntentDetector;

use Elastica\Client;

final class BrandIntentDetector
{
    private array $brandSet = [];      // ["nikon", "canon", ...]
    private array $brandOriginal = []; // ["nikon" ⇒ "Nikon", ...]

    public const BRAND_ALIASES = [
        'fuji' ⇒ 'Fujifilm', 'pana' ⇒ 'Panasonic',
        'think tank' ⇒ 'ThinkTank',
        'cannon' ⇒ 'Canon', 'kanon' ⇒ 'Canon',
        'nikkon' ⇒ 'Nikon', 'olimpus' ⇒ 'Olympus',
        'go pro' ⇒ 'GoPro', 'insta 360' ⇒ 'Insta360',
        'fuji film' ⇒ 'Fujifilm', 'glare one' ⇒ 'GlareOne',
        'peak desing' ⇒ 'Peak Design',
    ];

    public function __construct(private Client $esClient, private string $indexName) {}
```

```

public function detect(string $query): ?BrandIntent
{
    $this->ensureBrandsLoaded();
    $tokens = explode(' ', mb_strtolower(trim($query)));
    if (empty($tokens)) return null;

    // Sprawdź 2-wyrazowe marki, potem 1-wyrazowe
    foreach ([2, 1] as $n) {
        if ($n > count($tokens)) continue;
        $prefix = implode(' ', array_slice($tokens, 0, $n));
        $remainder = trim(implode(' ', array_slice($tokens, $n)));

        // Dokładne dopasowanie
        if (isset($this->brandOriginal[$prefix])) {
            return new BrandIntent($this->brandOriginal[$prefix], $remainder, false);
        }
        // Alias
        if (isset(self::BRAND_ALIASES[$prefix])) {
            return new BrandIntent(self::BRAND_ALIASES[$prefix], $remainder, true);
        }
    }
    return null;
}

public function rewriteQuery(string $query, BrandIntent $intent): string
{
    // "fuji xt5" → "Fujifilm xt5"
    if (!$intent->wasAlias) return $query;
    foreach (self::BRAND_ALIASES as $alias => $canonical) {
        if ($canonical === $intent->brand
            && stripos($query, $alias) !== false) {
            return str_ireplace($alias, $canonical, $query);
        }
    }
    return $query;
}

private function ensureBrandsLoaded(): void
{
    if (!empty($this->brandSet)) return;
    $index = $this->esClient->getIndex($this->indexName);
    $response = $index->search([
        'size' => 0,
        'aggs' => ['brands' => ['terms' => ['field' => 'producer', 'size' => 500]]]
    ]);
    foreach ($response->getAggregation('brands')['buckets'] as $b) {
        $key = $b['key'];
        $lower = mb_strtolower($key);
        $this->brandSet[$lower] = true;
        $this->brandOriginal[$lower] = $key;
    }
}
}

```

Plik 3: CategoryIntent.php — Value Object

```

<?php
declare(strict_types=1);
namespace App\Component\Elasticsearch\IntentDetector;

final class CategoryIntent
{
    public function __construct(
        public readonly array $subcategories, // ["lampe LED", "lampe studyjne LED"]
        public readonly string $remainder,    // "godox" (z "lampe godox")
    ) {}

    public function isSingleCategory(): bool

```



```

        { return count($this->subcategories) === 1; }

        public function hasRemainder(): bool
        { return '' !== $this->remainder; }
    }

```

Plik 4: CategoryIntentDetector.php

Mapowanie słów kluczowych na podkategorie z ES. To jest najważniejsza tablica — musi pasować do Twojego drzewa kategorii w Sylius.

```

<?php
declare(strict_types=1);
namespace App\Component\Elasticsearch\IntentDetector;

final class CategoryIntentDetector
{
    /**
     * Mapa: słowo kluczowe → lista podkategorii w ES.
     * DOSTOSUJ do swojego drzewa kategorii w Sylius!
     */
    public const CATEGORY_ALIASES = [
        'lampa' => ['lampy LED', 'lampy studyjne LED', 'lampy panelowe LED',
            'miecze świetlne LED', 'lampy pierścieniowe LED',
            'lampy błyskowe', 'lampy reporterskie'],
        'karta' => ['SD / SDHC / SDXC', 'CFexpress', 'microSD',
            'SD / SDHC', 'CompactFlash'],
        'karta sd' => ['SD / SDHC / SDXC', 'SD / SDHC'],
        'obiektyw' => ['obiektywy stałoogniskowe',
            'obiektywy zmienneogniskowe (zoom)',
            'obiektywy do lustrzanek',
            'obiektywy do bezlusterkowców'],
        'statyw' => ['statywy (trójnogi)', 'statywy do filmowania'],
        'filtr' => ['filtry', 'połówkowe i szare'],
        'akumulator' => ['akumulatory', 'baterie specjalistyczne'],
        'bateria' => ['akumulatory', 'baterie specjalistyczne'],
        'pasek' => ['paski', 'pasy biodrowe, szelki i kamizelki'],
        'klatka' => ['klatki', 'zestawy do foto-video'],
        'plecak' => ['plecaki fotograficzne'],
        'torba' => ['torby fotograficzne', 'torby kufry i walizki'],
        'softbox' => ['softboxy', 'softboxy oktagonalne', 'softboxy prostokątne'],
        'gimbal' => ['gimbale', 'stabilizatory', 'systemy stabilizacji'],
        'mikrofon' => ['mikrofony', 'mikrofony bezprzewodowe'],
        'dron' => ['drony'], 'drony' => ['drony'],
        'gopro' => ['kamery sportowe'],
        'ring light' => ['lampy pierścieniowe LED'],
        'miecz' => ['miecze świetlne LED'],
        'led panel' => ['lampy panelowe LED'],
        'tło' => ['tła kartonowe', 'tła składane', 'tła plastikowe',
            'tła materiałowe', 'tła winylowe'],
        'instax' => ['kompakty z natychmiastowym wydrukiem'],
        'lornetka' => ['uniwersalne', 'ornitologia i myślistwo dzienne',
            'turystyka', 'żeglarstwo', 'lornetki'],
        // Dodaj więcej aliasów dopasowanych do TWOJEGO drzewa kategorii
    ];

    public function detect(string $query): ?CategoryIntent
    {
        $q = mb_strtolower(trim($query));
        // Dokładne dopasowanie
        if (isset(self::CATEGORY_ALIASES[$q])) {
            return new CategoryIntent(self::CATEGORY_ALIASES[$q], '');
        }

        // Dopasowanie pierwszego/dwóch słów → reszta jako remainder
        $tokens = explode(' ', $q);
        foreach ([2, 1] as $n) {
            if ($n >= count($tokens)) continue;
            $prefix = implode(' ', array_slice($tokens, 0, $n));

```

```

        if (isset(self::CATEGORY_ALIASES[$prefix])) {
            $remainder = trim(implode(' ', array_slice($tokens, $n)));
            return new CategoryIntent(self::CATEGORY_ALIASES[$prefix], $remainder);
        }
    }
    return null;
}
}

```

Plik 5: `SearchIntent.php` — Kompozytowy Value Object

```

<?php
declare(strict_types=1);
namespace App\Component\Elasticsearch\IntentDetector;

final class SearchIntent
{
    public function __construct(
        public readonly string $originalQuery,
        public readonly string $normalizedQuery,
        public readonly ?BrandIntent $brand,
        public readonly ?CategoryIntent $category,
        public readonly ?string $focalLength,
        public readonly bool $usedCondition,
    ) {}

    public function hasBrandIntent(): bool { return null !== $this->brand; }
    public function hasCategoryIntent(): bool { return null !== $this->category; }
    public function hasFocalLengthIntent(): bool { return null !== $this->focalLength; }
}

```

Plik 6: `SearchIntentResolver.php` — Orkiestrator

```

<?php
declare(strict_types=1);
namespace App\Component\Elasticsearch\IntentDetector;

final class SearchIntentResolver
{
    private const RE_DIGIT_TO_ALPHA = '/(\d)([a-zA-Z])/';
    private const RE_MARK_ROMAN = '/\bmark\s+(i{1,4}v?|iv|v)\b/i';
    private const RE_FOCAL_LENGTH = '/\b(\d{2,3})\s*[\--]\s*(\d{2,3})\b/';
    private const USED_PREFIXES = ['używan', 'używany', 'używany', 'uży', 'uzy'];

    public const LENS_SUBCATS = [
        'obiektywy stałogniskowe', 'obiektywy zmienneogniskowe (zoom)',
        'obiektywy do lustrzanek', 'obiektywy do bezlusterkowców',
    ];
    public const MAIN_SUBCATS = [
        'aparaty cyfrowe', 'bezlusterkowce', 'kompakty', 'lustrzanki',
        'obiektywy stałogniskowe', 'obiektywy zmienneogniskowe (zoom)',
        'obiektywy do lustrzanek', 'obiektywy do bezlusterkowców',
        'kamery cyfrowe', 'kamery sportowe', 'drony',
    ];

    public function __construct(
        private BrandIntentDetector $brandDetector,
        private CategoryIntentDetector $categoryDetector,
    ) {}

    public function resolve(string $query): SearchIntent
    {
        $original = $query;

        // 1. "a7iv" → "a7 iv"
        $query = preg_replace(self::RE_DIGIT_TO_ALPHA, '$1 $2', $query) ?? $query;
        // 2. "mark III" → "markIII"
    }
}

```

```

$query = preg_replace_callback(self::RE_MARK_ROMAN,
    fn($m) => 'mark' . $m[1], $query) ?? $query;

$qLower = mb_strtolower(trim($query));

// 3. Brand
$brand = $this->brandDetector->detect($query);
// 4. Alias rewrite
if ($brand->wasAlias) {
    $query = $this->brandDetector->rewriteQuery($query, $brand);
}

// 5. Condition (used)
$usedCondition = false;
$queryForEs = $query;
foreach (explode(' ', $qLower) as $word) {
    foreach (self::USED_PREFIXES as $prefix) {
        if (str_starts_with($word, $prefix)) {
            $usedCondition = true;
            break 2;
        }
    }
}
if ($usedCondition) {
    $clean = [];
    foreach (explode(' ', $query) as $w) {
        $keep = true;
        foreach (self::USED_PREFIXES as $p) {
            if (str_starts_with(mb_strtolower($w), $p)) { $keep = false; break; }
        }
        if ($keep) $clean[] = $w;
    }
    $queryForEs = trim(implode(' ', $clean)) ?: $query;
}

// 6. Category
$catCheck = $brand ? $brand->remainder : $qLower;
$category = !empty($catCheck) ? $this->categoryDetector->detect($catCheck) : null;

// 7. Focal length
$focal = null;
if (!$category && preg_match(self::RE_FOCAL_LENGTH, $qLower, $m)) {
    $focal = $m[1] . '-' . $m[2];
}

return new SearchIntent($original, $queryForEs, $brand, $category, $focal, $usedCondition);
}
}

```

Krok 3: Zarejestruj serwisy

Utwórz plik **Resources/services/intent_detection.xml** :

```

<service id="app.elasticsearch.intent_detector.brand"
    class="App\Component\Elasticsearch\IntentDetector\BrandIntentDetector">
    <argument type="service" id="fos_elastica.client" />
    <argument>%bitbag_es_index_prefix%bitbag_shop_products_%kernel.environment%/</argument>
</service>

<service id="app.elasticsearch.intent_detector.category"
    class="App\Component\Elasticsearch\IntentDetector\CategoryIntentDetector" />

<service id="app.elasticsearch.intent_detector.resolver"
    class="App\Component\Elasticsearch\IntentDetector\SearchIntentResolver">
    <argument type="service" id="app.elasticsearch.intent_detector.brand" />
    <argument type="service" id="app.elasticsearch.intent_detector.category" />
</service>

```

Faza 4 — Nowy ProductsQueryBuilder

KLUCZOWA ZMIANA — zamiana prostego MultiMatch na intent-aware bool query

Obecny `ProductsQueryBuilder` używa jednego `MultiMatch`. Nowy buduje 3 warianty query zależnie od intencji:

Wariant 1: Category-intent ("lampa", "karta sd")

- Filter na podkategoriach (Terms filter)
- Jeśli remainder ("klatka sony a7 iv"): `MultiMatch` na remainder w ramach filtra
- Jeśli czyste ("lampa"): `MatchAll` (ranking przez `function_score`)

Wariant 2: Text-matching ("canon r8", "sony a7iv")

- `bool.should` z:
 - `MultiMatch` na name/description/producer/taxon (fuzziness + `min_should_match` 70%)
 - `match_phrase` z `slop=2` (`boost=50`, lub 10 gdy brand-intent)
 - `match_phrase_prefix` (`boost=5`, lub 2 gdy brand-intent)
 - Term matches na `ean.keyword`, `product_code.keyword` (`boost=100`)
- Brand filter (gdy brand-intent)
- Focal-length filter na obiektach (gdy wykryto ogniskową)

Wariant 3: Used-intent

- Hard filter: `{"term": {"condition": "used"}}` wstrzyknięty do `bool.filter`
- Phrase boost +500 na oczyszczonym zapytaniu (bez słowa "używany")

Krok 1: Zamień `ProductsQueryBuilder.php`

Plik gotowy jest w: [src/Component/Elasticsearch/QueryBuilder/ProductsQueryBuilder.php](#)

Kluczowe zmiany:

- Nowa zależność: `SearchIntentResolver` (3. argument konstruktora)
- Metoda `getLastIntent()` — udostępnia `SearchIntent` dla `FunctionScore`
- `buildCategoryIntentQuery()` — filter na podkategoriach
- `buildTextMatchingQuery()` — `bool.should` z `match_phrase` + exact codes

Krok 2: Zaktualizuj `query_builder.xml`

```
<!-- Dodaj SearchIntentResolver jako 3. argument -->
<service id="app.elasticsearch.query_builder.products"
        class="App\Component\Elasticsearch\QueryBuilder\ProductsQueryBuilder">
    <argument type="service" id="bitbag_sylius_elasticsearch_plugin.query_builder.is_enabled" />
    <argument type="service" id="bitbag_sylius_elasticsearch_plugin.query_builder.has_channel" />
    <argument type="service" id="app.elasticsearch.intent_detector.resolver" />
    <argument>%bitbag_es_fuzziness%</argument>
    <argument>%bitbag_es_property_state%</argument>
    <argument>%env(int:ELASTICSEARCH_NAME_BOOSTER)%</argument>
    <argument>%env(int:ELASTICSEARCH_DESCRIPTION_BOOSTER)%</argument>
    <argument>%env(int:ELASTICSEARCH_PRODUCER_BOOSTER)%</argument>
    <argument>%env(int:ELASTICSEARCH_MAIN_CATEGORY_BOOSTER)%</argument>
```

```
<argument>%env(int:ELASTICSEARCH_SUB_CATEGORY_BOOSTER)%</argument>  
<argument>%env(int:ELASTICSEARCH_CHILD_CATEGORY_BOOSTER)%</argument>  
</service>
```

Faza 5 — Nowy FunctionScoreProductsQueryBuilder

Dynamiczny scoring zależny od intencji

Było (5 functions, stałe):

Funkcja	Modifier	Weight
popularity	log1p	1
rating	none	1
new_or_promotion	none	1
featured	none	100
realization_time_booster	none	1.0

Nowe — 2 warianty scoringu:

Category-intent (ok. 8 functions):

Funkcja	Modifier	Weight
popularity	sqrt	50
rating	none	z .env
availability (in_stock)	filter	+30
bestseller	filter	+60
featured	none	z .env
new_or_promotion	none	+10
condition (used/new)	filter	+200/+25
realization_time_booster	zachowany	

Text-matching (ok. 13 functions):

Funkcja	Weight (brand)	Weight (bez brand)
brand match	+300	—
popularity (log1p)	80	80
price (log1p * 0.001)	30	15
availability (in_stock)	+50	+50
bestseller	+60	+60
main_subcats boost	+1000	+120
featured	z .env	z .env
new_or_promotion	z .env	z .env

promo	+20	+20
new products	+30	+30
condition	+50/+200	+25/+200
focal-length lens	+150	+150
realization_time_booster	zachowany	

Krok: Zamień `FunctionScoreProductsQueryBuilder.php`

Nowa metoda `buildIntentAwareQuery(SearchIntent)` jest wywoływana przez `SiteWideProductsQueryBuilder`.

Dodaj argument `bestsellerProperty` (10. pozycja w konstruktorze) w services XML.

Faza 6 — Synonimy i ekspansja zapytań

Rozszerzenie istniejących synonimów + query-time expansion

Poziom 1: Synonimy ES (w fos_elastica.yaml)

Rozszerz istniejącą listę synonimów o nowe wpisy:

```
synonym_graph_filter:
  type: synonym_graph
  synonyms:
    # Dwukierunkowe
    - "fuji, fujifilm, fuij, fuijfilm, fuji film"
    - "om system, olympus, olimpus"
    - "smallrig, smallring"
    - "canon, kannon, cannon, kanon"
    - "szkło, obiektyw"
    - "statyw, tripod, trójnóg, trojnóg"
    - "lampa, flesz, flash, błysk"
    - "karta pamięci, karta SD, karty SD, karty pamięci"
    - "etui, pokrowiec, case, futerał"
    - "dron, drone, quadcopter"
    # Jednokierunkowe
    - "lustrzanka ⇒ aparat cyfrowy"
    - "szklanka, szklanki ⇒ obiektyw"
    - "beczka ⇒ teleobiektyw"
    - "body, ciało ⇒ korpus"
    - "leika, lejka, lejca ⇒ leica"
    - "cage ⇒ klatka"
    - "strap ⇒ pasek"
    - "strobe ⇒ lampa studyjna"
```

Poziom 2: Query-time expansion (w PHP)

Dodaj do `SearchIntentResolver::resolve()` krok ekspansji synonimów:

1. **TYPO_CORRECTIONS** — popraw literówki: "obiektw" → "obiektyw"
2. **SLANG_ALIASES** — zamień slang: "szklanka" → "obiektyw"
3. **EN_PL_ALIASES** — dodaj polskie equivalenty: "cage" → dodaj "klatka" jako should
4. **SYNONYM_EXPANSIONS** — dwukierunkowe grupy: "dron" ↔ "drone" ↔ "quadcopter"

Limit: Max 5 dodatkowych fraz z ekspansji, aby nie spowolnić zapytań.

Faza 7 — Reranking sprzedaży

Post-ES reranking na podstawie danych sprzedażowych z ERP

Prototyp CyfroSearch stosuje post-ES reranking na podstawie danych z Verto ERP. Wdrożenie w PHP:

Formuła:

$$\text{final_score} = \text{es_score} * (1 + \log1p(\text{sales_30d}) * 0.15 + \log1p(\text{sales_365d}) * 0.05)$$

Krok 1: Utwórz serwis SalesReranker

```
<?php
declare(strict_types=1);
namespace App\Component\Elasticsearch\Reranker;

final class SalesReranker
{
    /** @var array<string, array{s30: int, s365: int}> */
    private array $salesData = [];

    public function __construct(private SalesDataProviderInterface $provider) {}

    /**
     * @param array<int, array{id: int, score: float}> $esResults
     * @return array<int, array{id: int, score: float}>
     */
    public function rerank(array $esResults): array
    {
        if (empty($this->salesData)) {
            $this->salesData = $this->provider->loadAll();
        }

        foreach ($esResults as &$result) {
            $sku = (string) $result['id'];
            $sales = $this->salesData[$sku] ?? ['s30' => 0, 's365' => 0];
            $boost = 1 + log1p($sales['s30']) * 0.15
                + log1p($sales['s365']) * 0.05;
            $result['score'] *= $boost;
        }
        unset($result);

        usort($esResults, fn($a, $b) => $b['score'] <=> $a['score']);
        return $esResults;
    }
}
```

Krok 2: Dane sprzedażowe

Stwórz `SalesDataProviderInterface` z implementacją pobierającą dane z Verto ERP (lub z cache Redis/plik CSV).

Źródło	Waga	Opis
sales_30d	0.15	Sprzedaż ostatnich 30 dni (silniejszy sygnał)
sales_365d	0.05	Sprzedaż roczna (stabilizacja)

Faza 8 — Testy i walidacja

Sprawdź czy wszystko działa na danych produkcyjnych

Krok 1: Unit testy IntentDetector

```
public function testBrandIntentFuji(): void
{
    $intent = $this->resolver->resolve('fuji xt5');
    $this->assertNotNull($intent->brand);
    $this->assertEquals('Fujifilm', $intent->brand->brand);
    $this->assertEquals('xt5', $intent->brand->remainder);
    $this->assertTrue($intent->brand->wasAlias);
}

public function testCategoryIntentLampa(): void
{
    $intent = $this->resolver->resolve('lampa');
    $this->assertNotNull($intent->category);
    $this->assertGreaterThan(3, count($intent->category->subcategories));
    $this->assertEquals('', $intent->category->remainder);
}

public function testConditionIntentUsed(): void
{
    $intent = $this->resolver->resolve('używany canon r5');
    $this->assertTrue($intent->usedCondition);
    $this->assertStringNotContainsString('używany', $intent->normalizedQuery);
}
```

Krok 2: Testy integracyjne — kluczowe zapytania

Uruchom poniższe zapytania na indeksie produkcyjnym i sprawdź top 5 wyników:

Ref. Zapytania testowe

#	Zapytanie	Oczekiwany wynik	Co rozwiązuje
1	canon	Aparaty Canon na górze	Brand +300, main_cat +1000
2	fuji xt5	Fujifilm X-T5 #1	Brand alias + match_phrase
3	lampa	Wszystkie lampy LED/flash	Category-intent → 7 podkategorii
4	karta sd	Karty SD/SDHC/SDXC	Category alias, NIE obiektywy SDM
5	sony a7iv	Sony A7 IV #1	Model split: a7iv → a7 iv
6	canon eos r8	Canon EOS R8 #1	match_phrase slop=2
7	24-70	Obiektywy 24-70mm	Focal-length → lens filter
8	używany canon r5	Używane Canon R5	Condition hard filter + phrase
9	peak design paski	Paski Peak Design	Brand + category combo
10	klatka sony a7 iv	Klatki do Sony A7 IV	Category + remainder

11	obiektyw do canon r6	Obiektywy (RF mount, różne marki)	PATH A: keep cat, no brand filter
12	akumulator do canon r6	Akumulatory LP-E6NH	PATH B: cancel cat, brand filter
13	cage	Klatki operatorskie	EN→PL synonym
14	szklanka	Obiektywy	Slang alias
15	nikon z5 ii	Nikon Z5II #1	Roman merge: z5 ii → z5ii

Ref. Mapa plików — co dodać, co zmienić

NOWE pliki (8):

Plik	Faza	Opis
IntentDetector/BrandIntent.php	3	Value Object: brand, remainder, wasAlias
IntentDetector/BrandIntentDetector.php	3	Detekcja marki + aliasy
IntentDetector/CategoryIntent.php	3	Value Object: subcategories, remainder
IntentDetector/CategoryIntentDetector.php	3	Mapa keyword → podkategorie
IntentDetector/SearchIntent.php	3	Kompozytowy VO: brand + category + focal + used
IntentDetector/SearchIntentResolver.php	3	Orkiestrator: normalize → detect → compose
Resources/services/intent_detection.xml	3	Symfony DI definicje
Reranker/SalesReranker.php	7	Post-ES reranking z danymi ERP

ZMIENIONE pliki (5):

Plik	Faza	Co się zmienia
QueryBuilder/ProductsQueryBuilder.php	4	Intent-aware bool query (3 warianty)
QueryBuilder/FunctionScoreProductsQueryBuilder.php	5	Nowa metoda <code>buildIntentAwareQuery()</code>
QueryBuilder/SiteWideProductsQueryBuilder.php	5	Wywołanie <code>buildIntentAwareQuery()</code>
Resources/services/query_builder.xml	4-5	Nowe argumenty DI
Controller/Action/Api/ListProductsByPartialNameAction.php	4	Sugestie produktów + kategorii — obsługa <code>normalizedQuery</code> z <code>SearchIntent</code>

NIEZMODYFIKOWANE pliki (zachowane):

- Wszystkie PropertyBuildery (18 plików) — bez zmian
- `TaxonProductsQueryBuilder` — strony kategorii, bez zmian
- Wszystkie Findery (10 plików) — bez zmian
- Facety, Fabryki — bez zmian
- `SearchAction`, `SaleAction` — bez zmian
- `realization_time_booster` — ZACHOWANY we wszystkich ścieżkach

Ref. Docelowa konfiguracja fos_elastica.yaml

```
fos_elastica:
  clients:
    default: { url: '%env(ELASTICSEARCH_URL)%' }
  indexes:
    bitbag_shop_product:
      index_name: "%bitbag_es_index_prefix%bitbag_shop_products_%kernel.environment%"
      settings:
        analysis:
          char_filter:
            mark_normalizer:
              type: pattern_replace
              pattern: "\\bmark\\s+(i{1,4}v?|iv|v)\\b"
              replacement: "mark$1"
              flags: "CASE_INSENSITIVE"
          filter:
            polish_stem:
              type: stemmer
              language: polish
            icu_folding_filter:
              type: icu_folding
            edge_ngram_filter:
              type: edge_ngram
              min_gram: 2
              max_gram: 15
            synonym_graph_filter:
              type: synonym_graph
              synonyms:
                - "fuji, fujifilm, fuij, fuijfilm, fuji film"
                - "om system, olympus, olimpus"
                - "smallrig, smallring"
                - "canon, kannon, cannon, kanon"
                - "karty pamięci, karta pamięci, karta SD, karty SD"
                - "szkło, obiektyw"
                - "statyw, tripod, trójnóg"
                - "lampa, flesz, flash, błysk"
                - "etui, pokrowiec, case, futerał"
                - "lustrzanka ⇒ aparat cyfrowy"
                - "leika, lejka, lejca ⇒ leica"
                - "szklanka, szklanki ⇒ obiektyw"
                - "body ⇒ korpus"
                - "cage ⇒ klatka"
                - "strap ⇒ pasek"
          analyzer:
            polish_full:
              type: custom
              char_filter: [mark_normalizer]
              tokenizer: standard
              filter: [lowercase, polish_stem, icu_folding_filter, synonym_graph_filter]
            polish_prefix:
              type: custom
              tokenizer: standard
              filter: [lowercase, icu_folding_filter, edge_ngram_filter]
            polish_folded:
              type: custom
              char_filter: [mark_normalizer]
              tokenizer: standard
              filter: [lowercase, icu_folding_filter]
            index.mapping.total_fields.limit: 5000
      properties:
        enabled: ~
        name_pl_pl:
          type: text
          analyzer: polish_full
      fields:
        keyword: { type: keyword }
```

```
    sort: { type: icu_collation_keyword, language: pl, country: PL }
    prefix: { type: text, analyzer: polish_prefix }
    folded: { type: text, analyzer: polish_folded }
    morfologik: { type: text, analyzer: polish_full }
description_pl_pl:
  type: text
  analyzer: polish_full
producer:
  type: keyword
brand:
  type: keyword
ean:
  type: keyword
product_code:
  type: keyword
erp_id:
  type: keyword
subcategory:
  type: keyword
availability:
  type: keyword
condition:
  type: keyword
is_bestseller:
  type: boolean
is_promo:
  type: boolean
is_new:
  type: boolean
has_image:
  type: boolean
price:
  type: float
```

Podsumowanie faz

Faza	Opis	Wymaga reindexu?
0	Zmiana .env (boostery)	NIE
1	Pluginy ES + analizery polskie	TAK
2	Nowe pola w indeksie	TAK
3	Intent Detection (6 plików PHP)	NIE
4	Nowy ProductsQueryBuilder	NIE
5	Nowy FunctionScore	NIE
6	Synonimy + query expansion	TAK (synonimy ES)
7	Reranking sprzedażowy	NIE
8	Testy i walidacja	NIE

Rekomendowana kolejność wdrażania:

Fazy 0 → 1+2 (razem, wymagają reindexu) → 3 → 4+5 (razem) → 6 → 7 → 8

Quick win: Faza 0 (15 min) + Faza 3+4+5 (bez reindexu) dają największy skok jakości.